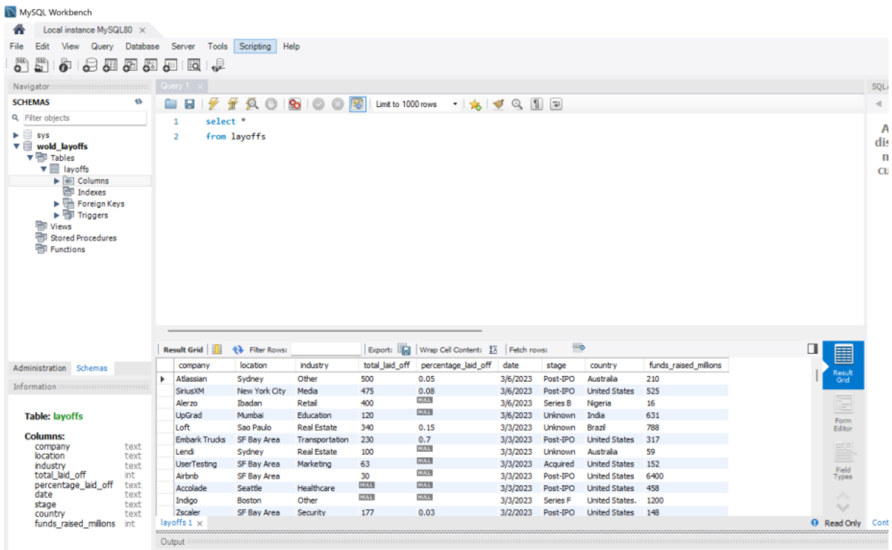
SQL Data Cleaning Project

SELECT*

FROM layoffs;



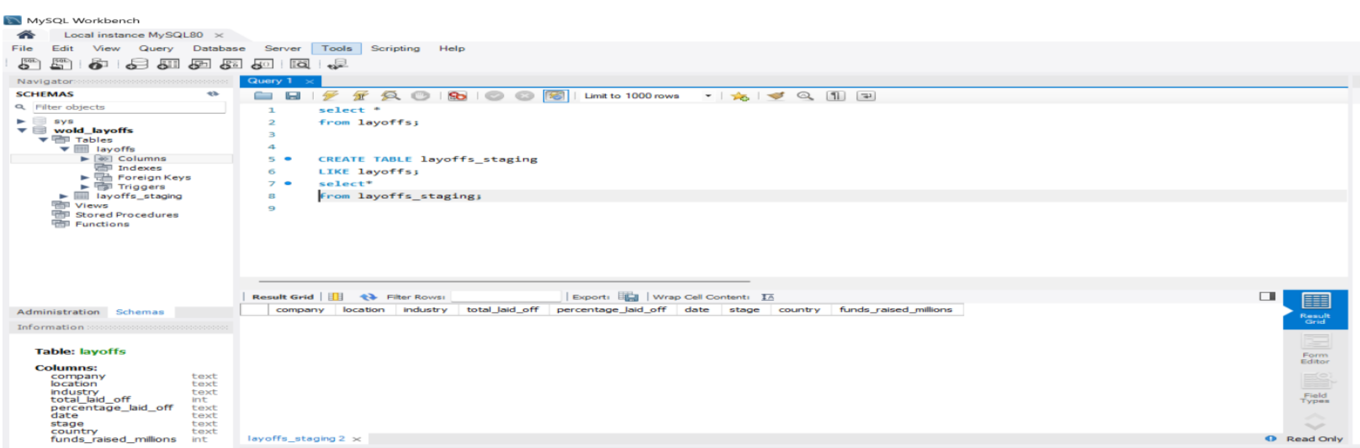
- **Prior to making any changes, I create a backup copy of the original table.**

CREATE TABLE layoffs_staging

LIKE layoffs;

SELECT *

FROM layoffs_staging;

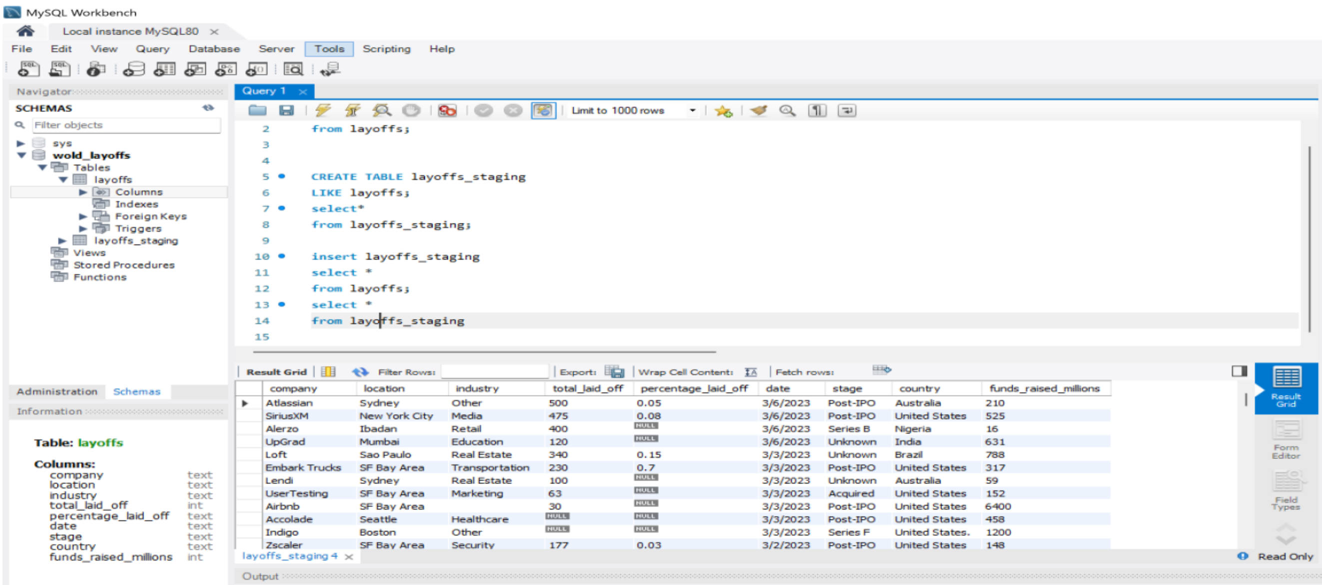


INSERT layoffs_staging SELECT *

FROM layoffs;

SELECT *

FROM layoffs_staging;



Detail:

- Using the same structure as layoffs, I make a new table called layoffs_staging.
- After that, I move all of the layoffs data into layoffs_staging.
- Lastly, I verify that the data has been accurately copied.

➢ **Remove Duplicates**

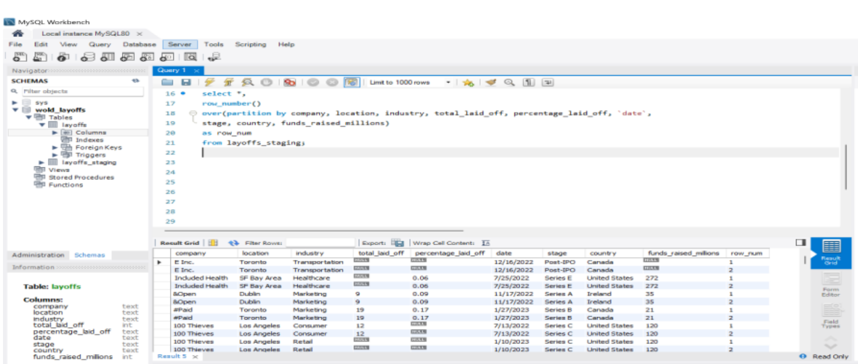
I use the ROW_NUMBER() function. SELECT *,

ROW_NUMBER()

OVER(PARTITION BY company, location, industry, total_laid_off, percentage_laid_off, `date`,

stage, country, funds_raised_millions) AS row_num

FROM layoffs_staging;



```
SELECT* , ROW_NUMBER()

OVER(PARTITION BY company, location, industry, total_laid_off, percentage_laid_off, `date` ,

stage, country, funds_raised_millions ) AS row_num FROM layoffs_staging)

SELECT*

FROM duplicate_cte WHERE row_num >1 ;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company	location	industry	total_laid_off	percentage_laid_off	date	stage	country	funds_raised_millions	row_num
Casper	New York City	Retail	NULL	NULL	9/14/2021	Post-IPO	United States	339	2
Cazoo	London	Transportation	750	0.15	6/7/2022	Post-IPO	United Kingdom	2000	2
Hibob	Tel Aviv	HR	70	0.3	3/30/2020	Series A	Israel	45	2
Wildlife Studios	Sao Paulo	Consumer	300	0.2	11/28/2022	Unknown	Brazil	260	2
Yahoo	SF Bay Area	Consumer	1600	0.2	2/9/2023	Acquired	United States	6	2

- To get rid of duplicates, make a new table called layoffs_staging2 to hold the cleaned data.
- Make a copy of every record in layoffs_staging, giving duplicates a row number.
- Remove any additional duplicate records.

```
CREATE TABLE `layoffs_staging2` (

`company` text,

`location` text,

`industry` text,

`total_laid_off` int DEFAULT NULL,

`percentage_laid_off` text,

`date` text,

`stage` text,

`country` text,

`funds_raised_millions` int DEFAULT NULL,

`row_num` INT

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

SELECT *

FROM layoffs_staging2; INSERT INTO layoffs_staging2

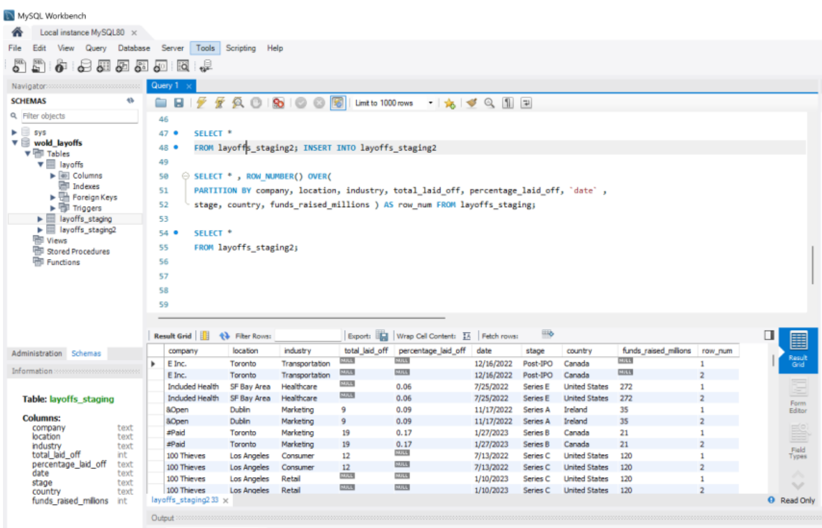
SELECT * , ROW_NUMBER() OVER(

PARTITION BY company, location, industry, total_laid_off, percentage_laid_off, `date` ,

stage, country, funds_raised_millions) AS row_num FROM layoffs_staging;

SELECT *

FROM layoffs_staging2;

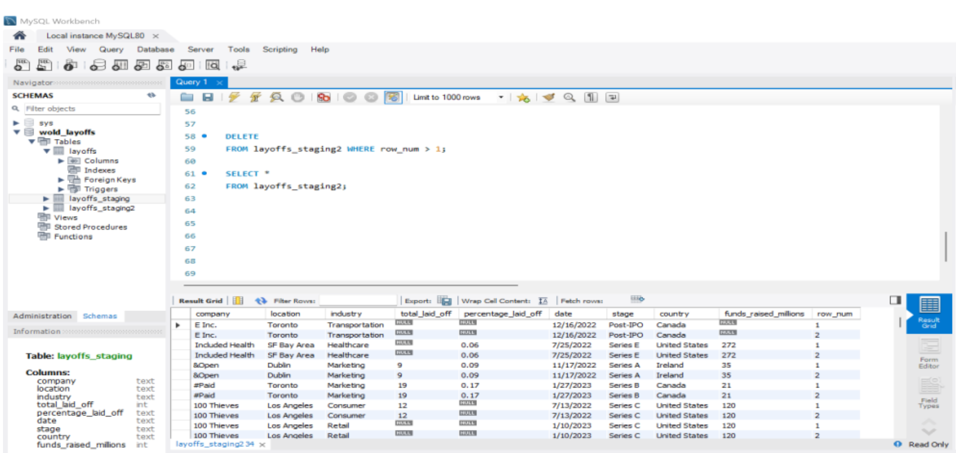


DELETE

FROM layoffs_staging2 WHERE row_num > 1;

SELECT *

FROM layoffs_staging2;



Detail:

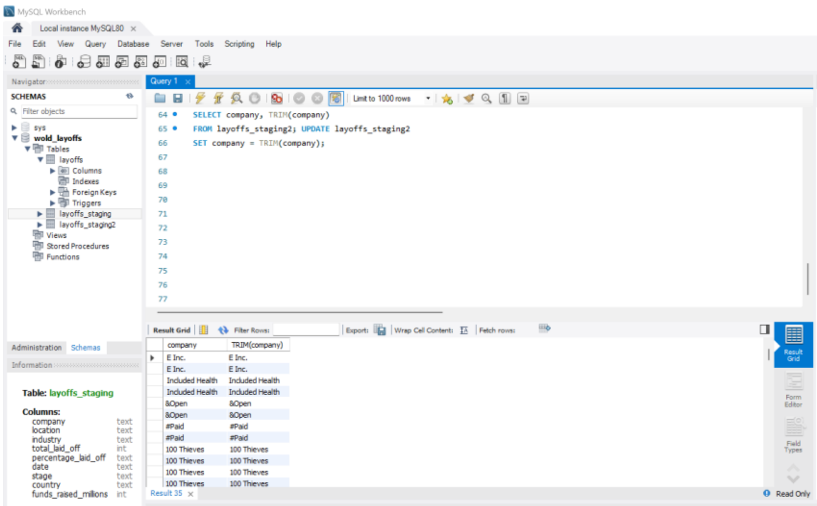
- I make layoffs_staging2, a second staging table.
- Duplicate rows are given a number using the ROW_NUMBER() method.
- I maintain only the first instance of each duplication and remove all rows where row_num > 1.

- **Standardize Data**

Remove white space from the company coloumn and update it SELECT company, TRIM(company)

FROM layoffs_staging2; UPDATE layoffs_staging2

SET company = TRIM(company);

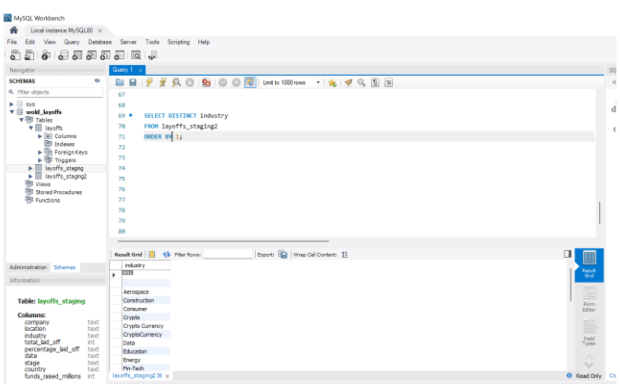


- **Verify and fix any spelling mistakes in the industry table. CHOOSE A UNIQUE INDUSTRY**

Select DISTINCT Industry

FROM layoffs_staging2

ORDER BY 1;



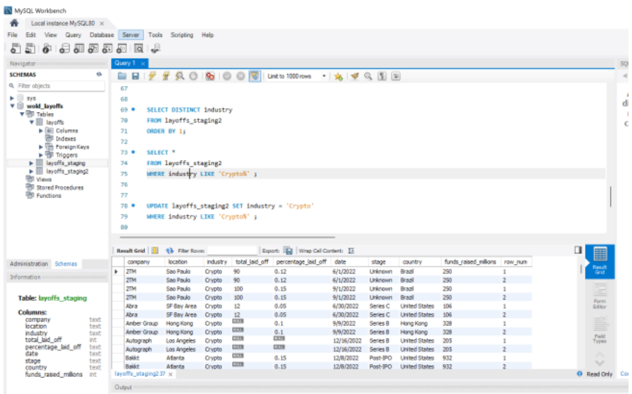
SELECT *

FROM layoffs_staging2

WHERE industry LIKE 'Crypto%' ;

UPDATE layoffs_staging2 SET industry = 'Crypto'

WHERE industry LIKE 'Crypto%' ;



- Analyzing for errors in the location table and correct where required.

SELECT DISTINCT location

FROM layoffs_staging2 ORDER BY 1;

UPDATE layoffs_staging2 SET country = 'United States'

WHERE country LIKE 'United States%' ;

Detail:

- TRIM(company): Removes extra spaces from the company names.
- LIKE 'Crypto%'

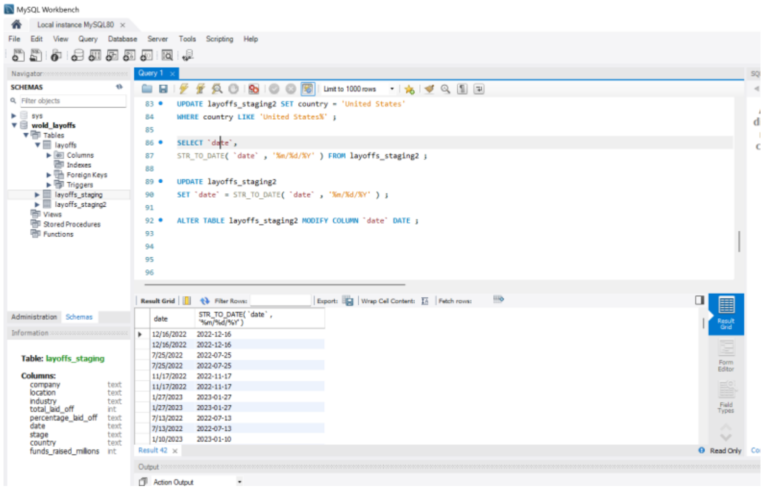
Step 4: Date Format

SELECT \`date\`,

STR_TO_DATE(\`date\` , '%m/%d/%Y') FROM layoffs_staging2 ;

UPDATE layoffs_staging2

SET \`date\` = STR_TO_DATE(\`date\` , '%m/%d/%Y') ;



Detail:

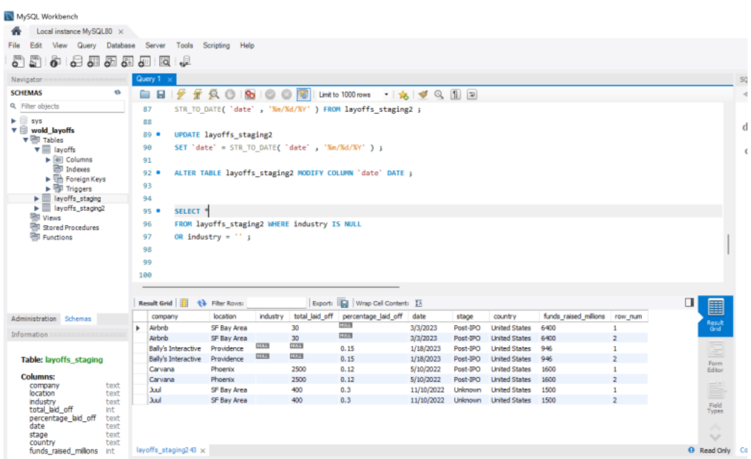
1. STR_TO_DATE() converts text-based dates into proper date format.
2. ALTER TABLE changes the column type to DATE.

- **Handle Missing Values**

--Find missing values in industry column: SELECT *

FROM layoffs_staging2 WHERE industry IS NULL

OR industry = '' ;

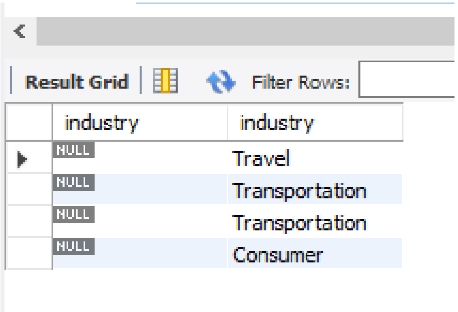


Replace blank values with NULL:

UPDATE layoffs_staging2

SET industry = null

WHERE industry = '' ;



industry	industry
NULL	Travel
NULL	Transportation
NULL	Transportation
NULL	Consumer

- **Fill missing industry names where possible:**

SELECT *

FROM layoffs_staging2 WHERE company = 'Airbnb'

SELECT t1.industry, t2.industry FROM layoffs_staging2 AS t1

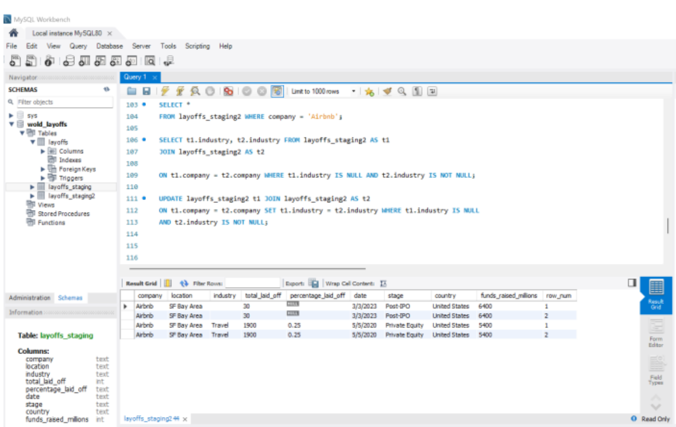
JOIN layoffs_staging2 AS t2

ON t1.company = t2.company WHERE t1.industry IS NULL AND t2.industry IS NOT NULL;

UPDATE layoffs_staging2 t1 JOIN layoffs_staging2 AS t2

ON t1.company = t2.company SET t1.industry = t2.industry WHERE t1.industry IS NULL

AND t2.industry IS NOT NULL;

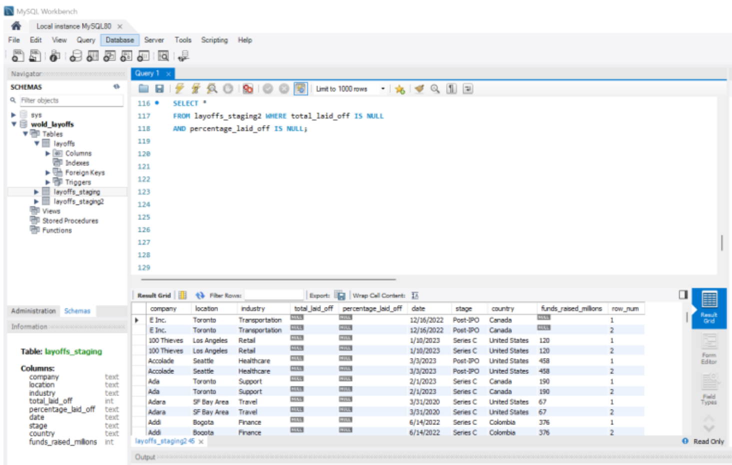


- **Remove rows where key fields are missing:**

SELECT *

FROM layoffs_staging2 WHERE total_laid_off IS NULL

AND percentage_laid_off IS NULL;



DELETE

FROM layoffs_staging2 WHERE total_laid_off IS NULL

AND percentage_laid_off IS NULL ;

- ➢ **Remove Columns which are not needed**

ALTER TABLE layoffs_staging2

DROP COLUMN row_num ;

Final Result

SELECT *

FROM layoffs_staging2 ;